StockPulse — Technical Requirements Document (TRD)

Product: StockPulse (inventory for warehouse / shop / online)
Document type: Technical requirements
Language: English

---

1. Purpose of this document

This TRD describes **how StockPulse is built and must behave** technically: frontend, backend, database choice, login, logging, hosting, version control, and core modules. It is based on the inventory idea and the work already done in the project.

---

2. Product summary (technical view)

StockPulse is a web inventory system for one business at a time (multi-tenant by business id). Owners and staff sign in with **WhatsApp OTP** (not Gmail login). Stock is tracked in **three locations**: warehouse, shop, and online. Owners see dashboard, prices, people, settings, and backups. Staff see limited screens by permission.

---

3. Technology stack

Area	Choice	Status
Frontend	**Next.js** (React, TypeScript)	Yes
Backend	**Next.js API routes** (Node.js / TypeScript)	Yes — same codebase as frontend
Database	**Firebase (Firestore)** — chosen database for the system	Documented in Schema PDF
Hosting	**Vercel**	Yes
Version control	**Git**	Yes
Messaging (OTP / alerts)	**WhatsApp** via Twilio for testing; later a production WhatsApp BSP	Testing now
Email (Resend)	Not used	**Skipped**
Custom DNS / domain A-record	No custom domain yet	**Skipped**

Languages used in the app code: TypeScript and JavaScript (Next.js / React / Node).
UI styling: Tailwind CSS with a fixed cream / navy palette (see Brand Kit PDF).

---

4. Frontend (Next.js)

4.1 Role

The frontend is the screens users see: landing, login, dashboard, items, add product, low stock, prices, staff/owners, settings.

4.2 Main app routes (pages)

Route	Who uses it	Purpose
`/`	Public	Landing
`/login`	Public / setup	WhatsApp OTP login and first-time business setup
`/dashboard`	Owners	Totals, alerts, price changes
`/items`	Owners + staff	Product list
`/add-product`	Owners + staff with “add”	Create product
`/low-stock`	Owners	Items below restock level
`/prices/[id]`	Owners	Price history, sale, purchase price
`/staff`	Owners	Owners and staff directory
`/settings`	Owners	Language, threshold, backup history, system status

4.3 Client gate

A **Gate** component checks session and role. Unauthenticated users go to `/login`. Staff cannot open owner-only routes. After login, each role has a home path (dashboard / add-product / items).

---

5. Backend (Node.js via Next.js)

5.1 Role

API routes under `app/api/` run on the server. They read/write data, send OTP, enforce access, run cron jobs, and talk to WhatsApp / Shopify when configured.

5.2 Important API groups

API area	Purpose

`/api/auth/otp`	Request and verify OTP; rate limit (max 3 requests / 10 minutes)
`/api/auth/logout`	End session
`/api/stock`	Products, sales, staff/owners actions (role-checked)
`/api/business`	Business setup and settings
`/api/system-status`	Owner-only health: DB, WhatsApp, last backup
`/api/health`	Simple health check
`/api/twilio/webhook`	Incoming WhatsApp messages
`/api/shopify`	Optional Shopify connect/sync
`/api/cron/*`	Daily backup, monthly chart, persist checks

Backend and frontend share one TypeScript project. There is **no separate Express server**; Node runs inside Next.js on Vercel.

---

6. Database — Firebase (chosen)

We document **Firebase** as the database for StockPulse.

6.1 Why Firebase

- Fits multi-tenant inventory data as **collections / documents** (one business, many products).
- Scales without managing a traditional server database yourself.
- **Firestore** holds structured business data.
- **Firebase Storage** (or equivalent blob storage) holds files such as backups and product photos.
- Works well with a **Next.js + Vercel** hosting model when the server uses Admin SDK for trusted writes.
- Security can be enforced with **server checks** (roles) and, where used, **Firestore Security Rules**.

6.2 What Firebase gives us (features we use / plan for)

Feature	Use for StockPulse
Cloud Firestore	Businesses, products, owners, staff, sales, alerts, sessions-related data as designed
Firebase Storage	Backup files, optional product images
Admin SDK (server)	Trusted API routes update data after role checks
Project console	Monitoring, indexes, rules

Auth for end users stays WhatsApp OTP through our own login APIs. We do **not** require Gmail / Google sign-in for shop owners and staff.

Full collections, fields, access control, and blob/storage notes are in **`04-SCHEMA.pdf`**.

---

7. Login and logging (both)

7.1 Login (authentication)

1. User opens `/login` and enters WhatsApp number.
2. Server sends a one-time code (OTP) on WhatsApp (Twilio in testing).
3. User enters the code; server creates a **session** (long-lived cookie; about 30 days).
4. If the business is not set up yet, setup runs on the same login area (business name, etc.).
5. Rate limit: **maximum 3 OTP requests in 10 minutes** per number.
6. Logout clears the session via `/api/auth/logout`.

7.2 Logging (errors and system visibility)

- Owner **System Status** (Settings) shows whether database and WhatsApp look configured and when the last backup ran.
- API failures return clear error messages to the UI where needed (e.g. validation, forbidden role).
- Backup history records **success / failed** with timestamp and error text when a backup fails.
- Cron and health endpoints support operational checks on Vercel.

Detailed test practice (unit, parallel, V&V) is in **`05-TESTING.pdf`**.

---

8. Hosting — Vercel

- The app is hosted on **Vercel**.
- Builds and deploys from the Git repository.
- Serverless / edge-friendly Next.js routes run the APIs.
- Scheduled jobs (cron) are configured for backups and related tasks.

---

9. Version control — Git

- Source of truth is a **Git** repository.
- Changes are committed locally; push to remote only when the owner asks.
- Secrets (Twilio keys, Firebase service account JSON, tokens) must **never** be committed to Git.

---

10. Access control (summary)

Role	Can do

Equal owner	Full app, manage owners, reset business, settings, prices, reports
Co-owner	Most owner features; limited owner-management rules
Staff (add)	Add products; items; not dashboard/settings/prices
Staff (view)	View items only

Enforcement is on both **UI Gate** and **API** (`requireAccess` / role helpers).

---

11. Core inventory rules (must not break)

- Quantities are **separate**: `warehouse`, `shop`, `online`. Total is the sum.
- Sales deduct from a chosen location.
- Low stock uses each product's restock threshold.
- Price history supports charts and "change vs last month".
- Two businesses must stay **isolated** (no cross-tenant data).

---

12. Integrations

Integration	Notes
WhatsApp	OTP and optional Q&A / alerts; Twilio for testing
Shopify	Optional connect and sync
Blob / file storage	Backups with date+time stamp; history in Settings

---

13. Explicitly out of scope (this pack)

- Custom domain / DNS A-record documentation
- Resend email
- Gmail / Google login module
- Clipping / transcribe product modules
- Paid-plan commercial PDF (skipped until decided)

---

14. Related PDFs

1. Master Requirements Document
2. PRD
3. Flow diagram — visual start-to-end flowchart
4. Brand kit — colors and typography in use
5. Schema — Firebase structure, models, access, storage

6. Testing — unit, parallel, verification & validation